浙江大學
ZHEJIANG UNIVERSITY

Logic and Computer Design Fundamentals

Chapter 2 –Combinational Logic Circuits(1)

Ma De (马德)

made@zju.edu.cn

2023

College of Computer Science, Zhejiang University

Course Outline

- Binary Logic and Gates
- Boolean Algebra
- Standard Forms

Binary Logic and Gates

- Binary Logic
  - Binary variables
    - take on one of two values, 1 and 0
      - A, B, y, z, or X_1 for now, RESET, START_IT, or ADD1 later
  - Logical operators
    - operate on binary values and binary variables.
    - Basic logical operators are the logic functions AND, OR and NOT.
- Logic gates
  - implement logic functions.
  - Basic circuits
- Boolean Algebra
  - a useful mathematical system for specifying and transforming logic functions.
- **We study Boolean algebra as a foundation for designing and analyzing digital systems!**

Logical Operations

- The three basic logical operations are:
  - AND
  - OR
  - NOT
- AND is denoted by a dot (·), ×, ∧, or even none.
- OR is denoted by a plus (+) or ∨, .
- NOT is denoted by an overbar (¯), a single quote mark (') after, or (~) before the variable.

Logical Operations

- **AND:** **is read "Z is equal to X AND Y."**
  Z = 1 if and only if X = 1 and Y = 1; otherwise Z = 0;
- **OR:** **is read "Z is equal to X OR Y."**
  Z = 1 if X = 1 or if Y = 1, or if both X =1 and Y = 1; Z=0 if and only is X =0 and Y = 0;
- **NOT:** **is read "Z is equal to NOT X."**
  If X =1, Z= 0; but if X =0, then Z =1

Operator Definitions

- Operations are defined on the values "0" and "1" for each operator:

AND	OR	NOT
$0 \cdot 0 = 0$	$0 + 0 = 0$	$\overline{0} = 1$
$0 \cdot 1 = 0$	$0 + 1 = 1$	$\overline{1} = 0$
$1 \cdot 0 = 0$	$1 + 0 = 1$	
$1 \cdot 1 = 1$	$1 + 1 = 1$	

- Note: The statement:
  - 1 + 1 = 2 (read "one <u>plus</u> one equals two")

  is not the same as
  - 1 + 1 = 1 (read "1 <u>or</u> 1 equals 1").

Truth Tables

- <u>Truth table</u> – a tabular listing of the values of a function for all possible combinations of values on its arguments
- Example: Truth tables for the basic logic operations:

AND		
X	Y	$Z = X \cdot Y$
0	0	0
0	1	0
1	0	0
1	1	1

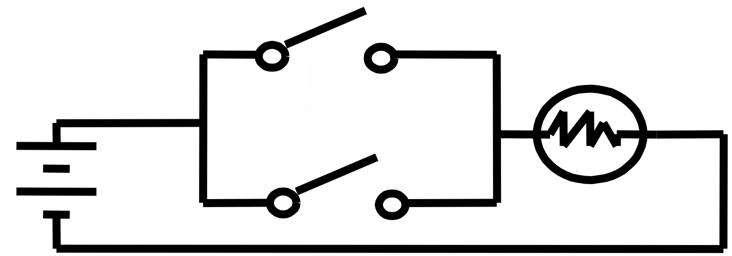
OR		
X	Y	$Z = X+Y$
0	0	0
0	1	1
1	0	1
1	1	1

NOT	
X	$Z=\overline{X}$
0	1
1	0

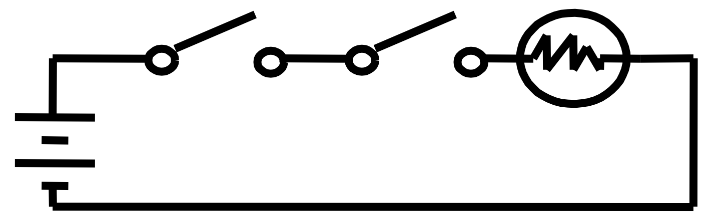
Logic Function Implementation

- Using Switches
  - For inputs:
    - logic 1 is switch closed
    - logic 0 is switch open
  - For outputs:
    - logic 1 is light on
    - logic 0 is light off.
  - NOT uses a switch such that:
    - logic 1 is switch open
    - logic 0 is switch closed

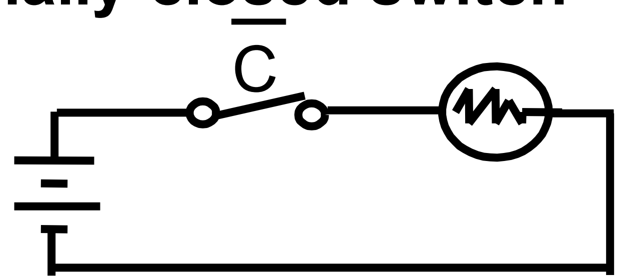
Switches in parallel => OR



Switches in series => AND

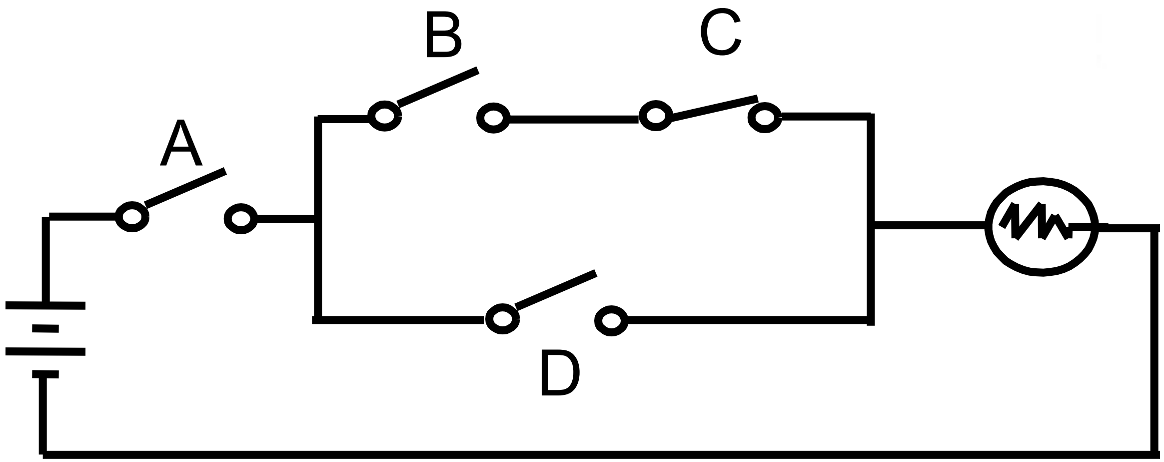


Normally-closed switch => NOT



Logic Function Implementation (Continued)

- **Example: Logic Using Switches**



- **Light is on (L = 1) for**

  L(A, B, C, D) = A ((B C) + D) = A B C + A D

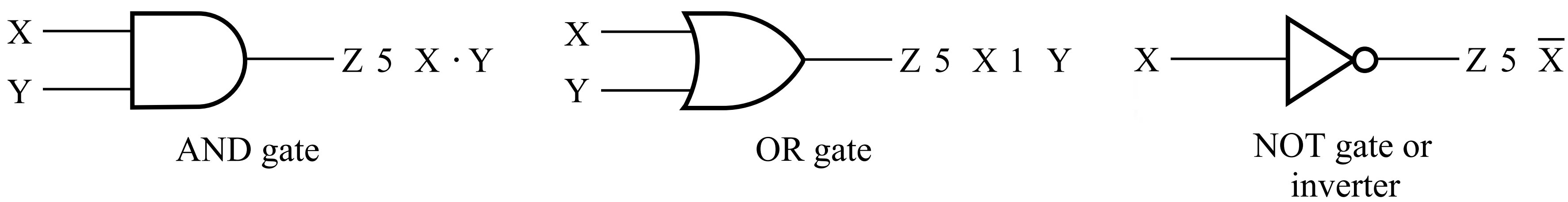
and off (L = 0), otherwise.

Logic Gates

- In the earliest computers, switches were opened and closed by magnetic fields produced by energizing coils in ***relays***. The switches in turn opened and closed the current paths.
- Later, ***vacuum tubes*** that open and close current paths electronically replaced relays.
- Today, ***transistors*** are used as electronic switches that open and close current paths.

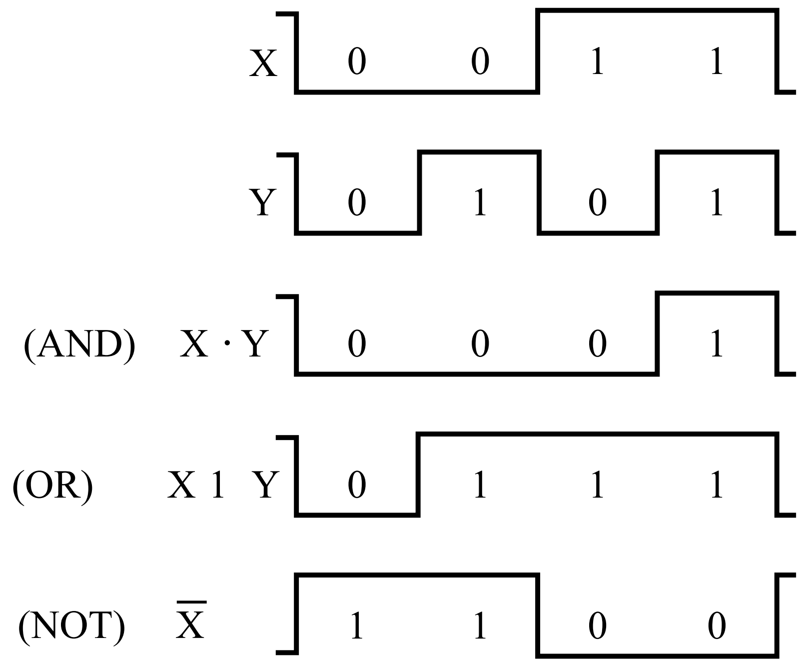
Logic Gate Symbols and Behavior

- Logic gates have special symbols:



(a) Graphic symbols

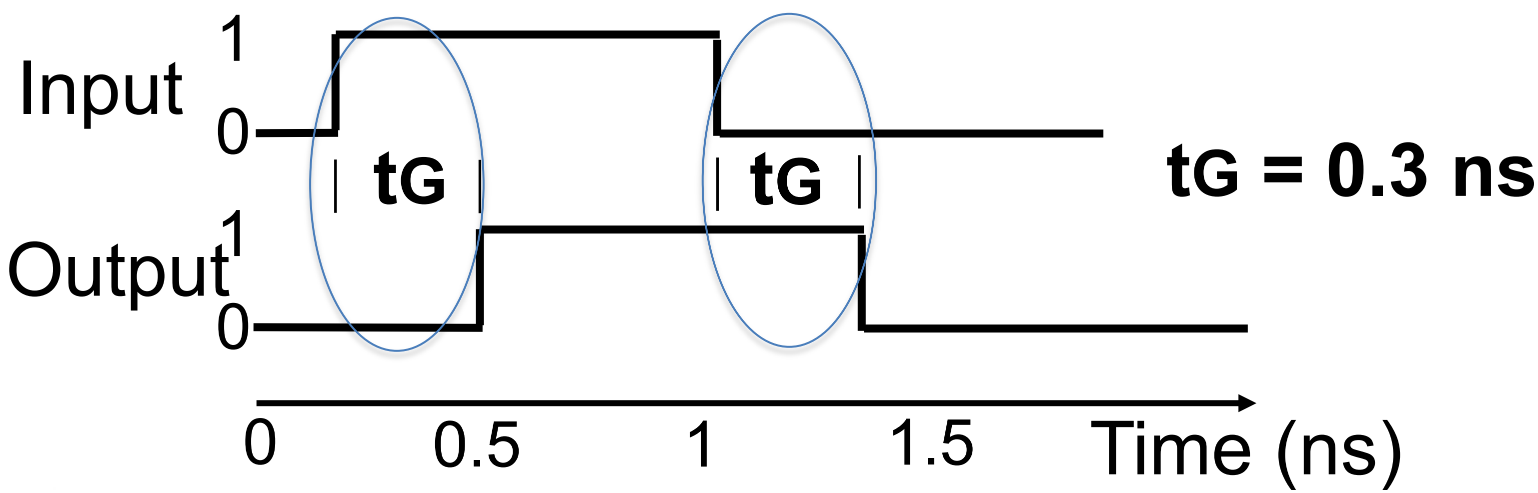
- And waveform behavior in time as follows:



(b) Timing diagram

Gate Delay

- In actual physical gates, if one or more input changes causes the output to change, the output change does not occur instantaneously.
- The delay between an input change(s) and the resulting output change is the *gate delay* denoted by t_G:



NOT AND (NAND)

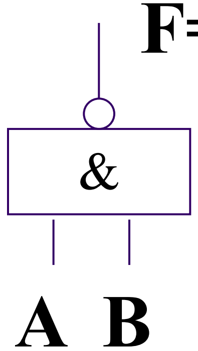
True and False

A	*B*	*L*
F	F	T
T	F	T
F	T	T
T	T	F

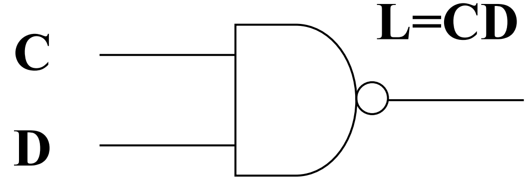
1 and 0

A	*B*	*L*
0	0	1
1	0	1
0	1	1
1	1	0

$$F=\overline{AB}$$



$$L=\overline{CD}$$



Graphic symbols

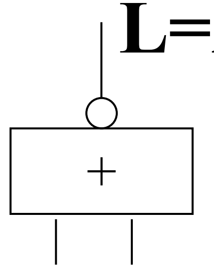
NOT OR (NOR)

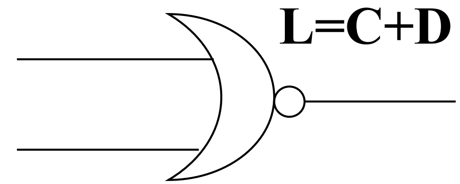
True and False

A	*B*	*L*
F	F	T
T	F	F
F	T	F
T	T	F

1 and 0

A	*B*	*L*
0	0	1
1	0	0
0	1	0
1	1	0

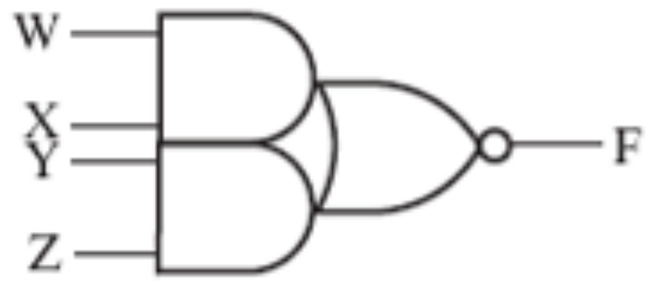




Graphic symbols

AND-OR-INVERT (AOI)

0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	0
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0



Graphic symbols

Exclusive-OR (XOR)

True and False

A	B	L
F	F	F
T	F	T
F	T	T
T	T	F

1 and 0

A	B	L
0	0	0
1	0	1
0	1	1
1	1	0

Graphic symbols

Eight Identities of XOR

$$X \oplus 0 = X \qquad X \oplus 1 = \overline{X}$$

$$X \oplus X = 0 \qquad X \oplus \overline{X} = 1$$

$$X \oplus \overline{Y} = \overline{X \oplus Y} \qquad \overline{X} \oplus Y = \overline{X \oplus Y}$$

$$X \oplus Y = Y \oplus X$$

$$(X \oplus Y) \oplus Z = X \oplus (Y \oplus Z) = X \oplus Y \oplus Z$$

$$X \oplus Y = X\overline{Y} + \overline{X}Y$$

浙江大学 ZheJiang University

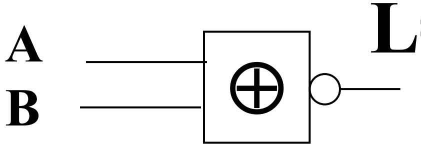
Exclusive-NOR (XNOR)

True and False

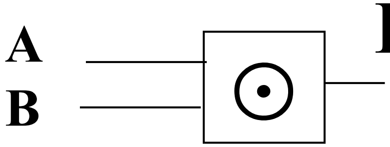
A	B	L
F	F	T
T	F	F
F	T	F
T	T	T

1 and 0

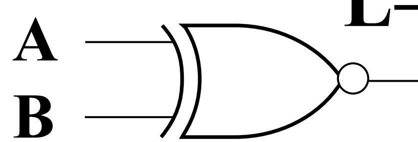
A	B	L
0	0	1
1	0	0
0	1	0
1	1	1



$$L=\overline{A\oplus B}$$



$$L=A\odot B$$

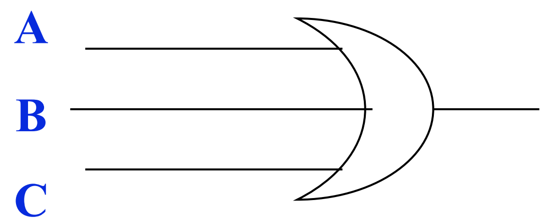


$$L=\overline{A\oplus B}$$

Graphic symbols

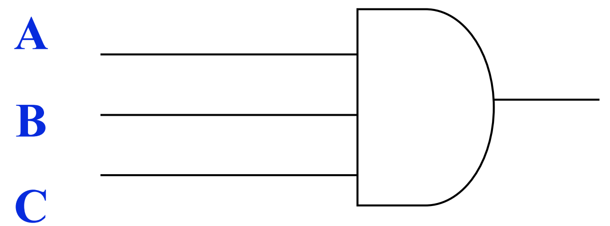
Multiple-input gates

OR:



F=A+B+C

AND:

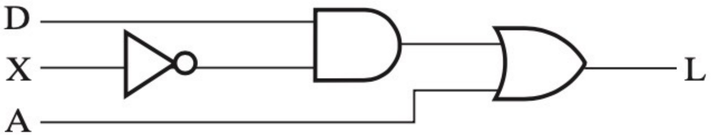


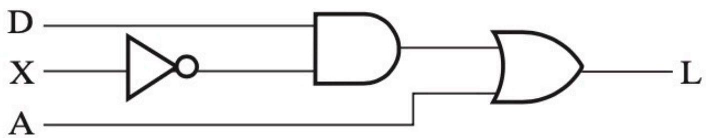
F=ABC

HDL-Verilog example

Truth Table for the Function $L = D\overline{X} + A$

D	X	A	L
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1






```
module fig2_5 (L, D, X, A);
    input D, X, A;
    output L;
    wire X_n, t2;

    not (X_n, X);
    and (t2, D, X_n);
    or (L, t2, A);
endmodule
```

Course Outline

- Binary Logic and Gates
- Boolean Algebra
- Standard Forms

Basic concepts of Boolean algebra

Boolean Algebra:

- also called Logic Algebra , useful mathematical system for specifying and transforming logic functions.
- a foundation for designing and analyzing digital systems!

Logical variables

- Is a binary variable.
- Two logical values only take 0,1 (true, false).
- The logic value not size

Three basic functions：

- **AND”、 “OR”、 “NOT”**

Boolean function defined

The definition of the logic function!!!

Assume a (circuit) input logic variable for $X_1, X_2, \ldots, X_n$, and the output logic variable is F. When $X_1, X_2, \ldots, X_n$ value is determined, the value of F is uniquely decided upon, then F is called to as $X_1, X_2, \ldots, X_n$, the logic function, denoted as

$$F=f(X_1, X_2, \ldots, X_n)$$

Note: Although X+Y=X+Z or XY=XZ ,

Y = Z is impossible true

Boolean algebra representation

- Boolean algebra representation: The Expression constituted by the binary variables, constants 0 and 1, logical operators, and parentheses

  For example: $F(X,Y,Z) = X + \overline{Y}Z$

  - Function can be expressed in a variety of expressions， Function expressions isn‘t unique.
- A single output and multi-output
  - the latter require more than one function expression represents the output.
- Can also use a truth table
  - A Boolean function is unique with a

X	Y	Z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Basic Equation

- Simplifying Boolean expressions with the properties

1.	$X + 0 = X$	2.	$X \cdot 1 = X$	
3.	$X + 1 = 1$	4.	$X \cdot 0 = 0$	
5.	$X + X = X$	6.	$X \cdot X = X$	
7.	$X + \overline{X} = 1$	8.	$X \cdot \overline{X} = 0$	
9.	$\overline{\overline{X}} = X$			
10.	$X + Y = Y + X$	11.	$XY = YX$	Commutative
12.	$(X + Y) + Z = X + (Y + Z)$	13.	$(XY)Z = X(YZ)$	Associative
14.	$X(Y + Z) = XY + XZ$	15.	$X + YZ = (X + Y)(X + Z)$	Distributive
16.	$\overline{X + Y} = \overline{X} \cdot \overline{Y}$	17.	$\overline{X \cdot Y} = \overline{X} + \overline{Y}$	DeMorgan's

Boolean function equal

- **Assuming two logical function**

$$F_1=f_1\ (X_1, X_2, \ldots, X_n)$$

$$F_2=f_2\ (X_1, X_2, \ldots, X_n)$$

If corresponding to X_1, X_2, ..., X_n for any set of values, the value of F1 and F2 are the same, then the functions F1 is equal to the function F2.

记作： $F_1 = F_2$

Truth table is completely identical!!!

Complementing and Duality rules

Complementing function

反函数

AND、OR swap，0、1 swap
& Put variable Inverses

For logic function F, interchange AND and OR operators ; complement each constant value and literal, then obtained the new function is the inverse function of the original function is referred to as: $\overline{F}$

For example： known $F=\overline{A}B+C\overline{D}$

According to inversion Rules can be obtained :

$$\overline{F}=(A+\overline{B})\cdot(\overline{C}+D)$$

Note the following two points:

① The holding operation priority unchanged, if necessary, add brackets indicate.

② Within converting, public non-operation remains unchanged for several variables

For example 1:

$$F = \overline{A} + \overline{B}\cdot(C + \overline{D}E),\text{则}$$

$$\overline{F} = A\cdot[B + \overline{C}(D + \overline{E})]$$

$$\overline{F} \neq A\cdot B + \overline{C}\cdot D + \overline{E}$$

Example 2:

$$L = A\cdot\overline{B + C + \overline{D}}$$

$$\overline{L} = \overline{A} + \overline{\overline{B}\cdot\overline{C}\cdot D}$$

Duality rules 对偶规则

AND、OR swap，0、1 swap
let variable unchanged

For logic function F, all "AND" changed to "OR"、"OR" changed "AND"、"0" changed to "1"、 "1" is changed to "0", then the resulting new logic function F' is F Duality function. Variable don't Inverses!!!

Seek a function F' Duality , the operation sequence keep as same as the original function 。

If F′ is the F Duality, then F is also F′ of Duality. F and F′ is mutually Duality formula .

If the two logical functions F and G are equal, then the Duality formula F' and G' are also equal.

Property of Duality of Boolean algebra

- Unless it happens to be self-dual, the dual of an expression does not equal the expression itself.
- Self-dual

  Example: $H = A \cdot B + A \cdot C + B \cdot C$

  dual $H = (A + B)(A + C)(B + C)$

  $= (A + BC)(B + C) = AB + AC + BC.$

So H is self-dual !

Other Useful Theorems

$x \times y + \bar{x} \times y = y$ $\quad (x+y)(\bar{x}+y) = y$ **Minimization**

$x + x \cdot y = x$ $\quad x \cdot (x+y) = x$ **Absorption**

$x + \bar{x} \times y = x + y$ $\quad x \times (\bar{x}+y) = x \times y$ **Simplification**

$$x \times y + \bar{x} \times z + y \times z = x \times y + \bar{x} \times z$$

$(x+y) \times (\bar{x}+z) \times (y+z) = (x+y) \times (\bar{x}+z)$ **Consensus**

$\overline{x+y} = \bar{x} \times \bar{y}$ $\quad \overline{x \times y} = \bar{x} + \bar{y}$ **DeMorgan's Laws**

Proof of Absorption Theorem

- $A + A \cdot B = A$ (Absorption Theorem)

Proof Steps | Justification (identity or theorem)

Proof Steps	Justification (identity or theorem)
$A + A \cdot B$	
$= A \cdot 1 + A \cdot B$	$X = X \cdot 1$
$= A \cdot (1 + B)$	$X \cdot Y + X \cdot Z = X \cdot (Y + Z)$(Distributive Law)
$= A \cdot 1$	$1 + X = 1$
$= A$	$X \cdot 1 = X$

- Compute the dual on both sides, the equation is equal either

$$A \cdot (A+B) = A$$

Proof of Minimization Theorem

- $AB + A\overline{B} = A$ (Minimization Theorem)

Proof Steps	Justification (identity or theorem)
$AB + A\overline{B}$	
$= A(B + \overline{B})$	$X \cdot Y + X \cdot Z = X \cdot (Y + Z)$ (Distributive Law)
$= A \cdot 1$	$1 + X = 1$
$= A$	

- Compute the dual on both sides, the equation is equal either

$$(A+B) \cdot (A+\overline{B}) = A$$

Proof of Simplification Theorem

- $A+\overline{A}B = A+B$ (Simplification Theorem)

Proof Steps — Justification (identity or theorem)

$A+\overline{A}B$

$= (A+\overline{A})(A+B)$ — $X+Y\cdot Z = (X+Y)\cdot(X+Z)$ (Distributive Law)

$= 1\cdot(A+B)$ — $X+\overline{X}=1$

$= A + B$

- Compute the dual on both sides, the equation is equal either

$$A(\overline{A}+B) = AB$$

Proof of Consensus Theorem

- $AB + \overline{A}C + BC = AB + \overline{A}C$ (Consensus Theorem)

Proof Steps **Justification (identity or theorem)**

$AB + \overline{A}C + BC$

$= AB + \overline{A}C + 1 \cdot BC$?

$= AB + \overline{A}C + (A + \overline{A}) \cdot BC$?

$AB + A'C + ABC + A'BC$ — $X(Y + Z) = XY + XZ$ (Distributive Law)

$= AB + ABC + A'C + A'BC$ — $X + Y = Y + X$ (Commutative Law)

$= AB \cdot 1 + ABC + A'C \cdot 1 + A'C \cdot B$ — $X \cdot 1 = X$, $X \cdot Y = Y \cdot X$ (Commutative Law)

$= AB(1 + C) + A'C(1 + B)$ — $X(Y + Z) = XY + XZ$ (Distributive Law)

$= AB \cdot 1 + A'C \cdot 1 = AB + A'C$ — $X \cdot 1 = X$

Proof of the 15^{th} basic identity

- **Using the absorption theorem**

  $$X + XY = X$$

- $$X + YZ = (X+Y)(X+Z)$$
  - From the right side

$$(X+Y)(X+Z) = X+XY+XZ+YZ$$
$$= X + XZ + YZ$$
$$= X + YZ$$

Example: Boolean Algebraic Proof

$$(\overline{X + Y})Z + X\overline{Y} = \overline{Y}(X + Z)$$

Proof Steps **Justification (identity or theorem)**

$$(\overline{X + Y})Z + X\overline{Y}$$

X' Y' Z + X Y'	(A + B)' = A' · B' (DeMorgan's Law)
= Y' X' Z + Y' X	A · B = B · A (Commutative Law)
= Y' (X' Z + X)	A(B + C) = AB + AC (Distributive Law)
= Y' (X' + X)(Z + X)	A + BC = (A + B)(A + C) (Distributive Law)
= Y' · 1 · (Z + X)	A + A' = 1
= Y' (X + Z)	1 · A = A, A + B = B + A (Commutative Law)

Substitution rules 替换规则

Any logical equation that contains a variable A, and if all occurrences of A's position are replaced with a logical function F, the equation still holds 。

Example

Assume: $X(Y+Z)=XY+XZ$， if X + YZ

Instead of X， then equation still holds：

$$(X+YZ)\,(Y+Z)=(X+YZ)Y+(X+YZ)Z$$

- Similarly equation (complementary)

$$f\,(X_1, X_2, \ldots, X_n)+\overline{f}\,(X_1, X_2, \ldots, X_n)=1$$

Additional equation

Shannon formula

- **Equation 1:**
  - Assuming: Function F contained variables x、$\bar{x}$, at "x AND F" operation, variable x may be replaced by "1", variable $\bar{x}$ can be replaced by "0". at "$\bar{x}$ AND F" operation, x can be "0", $\bar{x}$ can be replaced with "1".

$$x\,f(x, \bar{x}, y\, \dots, z_n) = x\ f(1, 0, y \dots, z)$$

$$\bar{x}\,f(x, \bar{x}, y\, \dots, z_n) = \bar{x}\ f(0, 1, y \dots, z)$$

- **This expression is actually expansion:**

$$\mathbf{X} \cdot \overline{\mathbf{X}} = 0 \text{ and } \mathbf{X{\cdot}X} = \mathrm{X}$$

Similarly, based on $A + \bar{A} = 1$, $A + \bar{A}B = A + B$, and $A + AB = A$, we can obtain the following equation

$$x + f(x, \bar{x}, y, \ldots, z) = x + f(0, 1, y, \ldots, z)$$

$$\bar{x} + f(x, \bar{x}, y, \ldots, z) = \bar{x} + f(1, 0, y, \ldots, z)$$

Additional **equation** is very useful for simplifying.

Example : $f = xy + x\bar{z} + (\bar{x} + \bar{y})(x + z)$, 求$x \cdot f$

$$\begin{aligned} x \cdot f &= x[xy + x\bar{z} + (\bar{x} + \bar{y})(x + z)] \\ &= x[1 \cdot y + 0 \cdot z + (0 + \bar{y})(1 + z)] \\ &= x(y + \bar{y}) = x \end{aligned}$$

◆**Equation 2:** The function F contains the both of the variables x、$\bar{x}$， may be follow:

$$F(X,\bar{X},Y\ldots Z)= x\cdot f(x,\bar{x},y,\ldots z)+\bar{x}f(x,\bar{x},y\ldots z)$$

$$= xf(1,0,y,\ldots z)+\bar{x}f(0,1,y\ldots z)$$

逻辑函数分解

(Problrm 3-2)

◆The same duality formula:

$$F(x,\bar{x},y,..z)=[x+f(1,0,y...z)]\cdot[\bar{x}+f(0,1,y,...z)]$$

Shannon expansion

Equation 1and Equation 2 can be obtained:：

$$\begin{aligned} f(x_1,x_2,x_3)=&x_1x_2x_3f(1,1,1)+x_1x_2\bar{x}_3f(1,1,0)\\ &+x_1\bar{x}_2x_3f(1,0,1)+x_1\bar{x}_2\bar{x}_3f(1,0,0)\\ &+\bar{x}_1x_2x_3f(0,1,1)+\bar{x}_1x_2\bar{x}_3f(0,1,0)\\ &+\bar{x}_1\bar{x}_2x_3f(0,0,1)+\bar{x}_1\bar{x}_2\bar{x}_3f(0,0,0) \end{aligned}$$

Expression Simplification

- Target: the smallest number of <u>literals</u> and the smallest number of terms (complemented and uncomplemented variables):
- AND-OR forms

$$A\,B + \overline{A}\,C\,D + \overline{A}\,B\,D + \overline{A}\,C\,\overline{D} + A\,B\,C\,D$$

$$= AB + ABCD + \overline{A}\,C\,D + \overline{A}\,C\,\overline{D} + \overline{A}\,B\,D$$

$$= AB + AB(CD) + \overline{A}\,C\,(D + \overline{D}) + \overline{A}\,B\,D$$

$$= AB + \overline{A}\,C + \overline{A}\,B\,D = B(A + \overline{A}D) + \overline{A}C$$

$$= B\,(A + D) + \overline{A}\,C \quad \text{5 literals}$$

Expression Simplification

- **OR-AND forms**

$$F = (\overline{A} + \overline{B})(\overline{A} + \overline{C} + D)(A + C)(B + \overline{C})$$

- **Dual**

$$F' = \overline{A}\,\overline{B} + \overline{A}\,\overline{C}D + AC + B\overline{C}$$

$$= (\overline{A}\,\overline{B} + B\overline{C} + \overline{A}\,\overline{C}D) + AC$$

$$= \overline{A}\,\overline{B} + B\overline{C} + AC$$

- **Dual of *F'* => F**

$$F = F'' = (\overline{A} + \overline{B})(B + \overline{C})(A + C)$$

Course Outline

- Binary Logic and Gates
- Boolean Algebra
- Standard Forms

Overview – Canonical Forms

- What are Canonical Forms?
- Minterms and Maxterms
- Index Representation of Minterms and Maxterms
- Sum-of-Minterm (SOM) Representations
- Product-of-Maxterm (POM) Representations
- Representation of Complements of Functions
- Conversions between Representations

Canonical Forms

- It is useful to specify Boolean functions in a form that:
  - Allows comparison for equality.
  - Has a correspondence to the truth tables
- Canonical Forms in common usage:
  - Sum of Minterms (SOM)
  - Product of Maxterms (POM)
  - Any function can be expressed to the canonical form

Minterms

- <u>Minterms</u> are AND terms with every variable present in either true or complemented form.
- Given that each binary variable may appear normal (e.g. x) or complemented (e.g. x'), there are 2^n minterms for n variables.

X	Y	Z	Product Term	Symbol	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
0	0	0	$\overline{X}\,\overline{Y}\,\overline{Z}$	m_0	1	0	0	0	0	0	0	0
0	0	1	$\overline{X}\,\overline{Y}Z$	m_1	0	1	0	0	0	0	0	0
0	1	0	$\overline{X}Y\overline{Z}$	m_2	0	0	1	0	0	0	0	0
0	1	1	$\overline{X}YZ$	m_3	0	0	0	1	0	0	0	0
1	0	0	$X\overline{Y}\,\overline{Z}$	m_4	0	0	0	0	1	0	0	0
1	0	1	$X\overline{Y}Z$	m_5	0	0	0	0	0	1	0	0
1	1	0	$XY\overline{Z}$	m_6	0	0	0	0	0	0	1	0
1	1	1	XYZ	m_7	0	0	0	0	0	0	0	1

Minterms

- Complemented or uncomplemented? A literal is
  - complemented if the corresponding bit is 0
  - Uncomplemented if the corresponding bit 1
- The <u>index</u>: the decimal equivalent of the binary combination corresponding to the Minterm

X	Y	Z	Product Term	Symbol	m_0	m_1	m_2	m_3	m_4	m_5	m_6	m_7
0	0	0	$\overline{X}\,\overline{Y}\,\overline{Z}$	m_0	1	0	0	0	0	0	0	0
0	0	1	$\overline{X}\,\overline{Y}Z$	m_1	0	1	0	0	0	0	0	0
0	1	0	$\overline{X}Y\overline{Z}$	m_2	0	0	1	0	0	0	0	0
0	1	1	$\overline{X}YZ$	m_3	0	0	0	1	0	0	0	0
1	0	0	$X\overline{Y}\,\overline{Z}$	m_4	0	0	0	0	1	0	0	0
1	0	1	$X\overline{Y}Z$	m_5	0	0	0	0	0	1	0	0
1	1	0	$XY\overline{Z}$	m_6	0	0	0	0	0	0	1	0
1	1	1	XYZ	m_7	0	0	0	0	0	0	0	1

Maxterms

- Maxterms are OR terms with every variable in true or complemented form.
- Given that each binary variable may appear normal (e.g., x) or complemented (e.g., x'), there are 2^n maxterms for n variables.

X	Y	Z	Sum Term	Symbol	M_0	M_1	M_2	M_3	M_4	M_5	M_6	M_7
0	0	0	$X+Y+Z$	M_0	0	1	1	1	1	1	1	1
0	0	1	$X+Y+\overline{Z}$	M_1	1	0	1	1	1	1	1	1
0	1	0	$X+\overline{Y}+Z$	M_2	1	1	0	1	1	1	1	1
0	1	1	$X+\overline{Y}+\overline{Z}$	M_3	1	1	1	0	1	1	1	1
1	0	0	$\overline{X}+Y+Z$	M_4	1	1	1	1	0	1	1	1
1	0	1	$\overline{X}+Y+\overline{Z}$	M_5	1	1	1	1	1	0	1	1
1	1	0	$\overline{X}+\overline{Y}+Z$	M_6	1	1	1	1	1	1	0	1
1	1	1	$\overline{X}+\overline{Y}+\overline{Z}$	M_7	1	1	1	1	1	1	1	0

Maxterms

- Complemented or uncomplemented? A literal is
  - complemented if the corresponding bit is 1
  - Uncomplemented if the corresponding bit is 0
- The <u>index</u>: the decimal equivalent of the binary combination corresponding to the Maxterm

X	Y	Z	Sum Term	Symbol	M_0	M_1	M_2	M_3	M_4	M_5	M_6	M_7
0	0	0	$X+Y+Z$	M_0	0	1	1	1	1	1	1	1
0	0	1	$X+Y+\overline{Z}$	M_1	1	0	1	1	1	1	1	1
0	1	0	$X+\overline{Y}+Z$	M_2	1	1	0	1	1	1	1	1
0	1	1	$X+\overline{Y}+\overline{Z}$	M_3	1	1	1	0	1	1	1	1
1	0	0	$\overline{X}+Y+Z$	M_4	1	1	1	1	0	1	1	1
1	0	1	$\overline{X}+Y+\overline{Z}$	M_5	1	1	1	1	1	0	1	1
1	1	0	$\overline{X}+\overline{Y}+Z$	M_6	1	1	1	1	1	1	0	1
1	1	1	$\overline{X}+\overline{Y}+\overline{Z}$	M_7	1	1	1	1	1	1	1	0

Minterm and Maxterm Relationship

- Review: DeMorgan's Theorem

  $\overline{x \cdot y} = \bar{x} + \bar{y}$ and $\overline{x + y} = \bar{x} \times \bar{y}$

- Two-variable example:

  $M_2 = \bar{x} + y$ and $m_2 = x \cdot \bar{y}$

  Thus M_2 is the complement of m_2 and vice-versa.

- Since DeMorgan's Theorem holds for *n* variables, the above holds for terms of *n* variables
- giving:

  $M_i = \overline{m_i}$ and $m_i = \overline{M_i}$

  Thus M_i is the complement of m_i.

Index	Minterm	Maxterm
0	$\bar{x}\,\bar{y}$	$x + y$
1	$\bar{x}\,y$	$x + \bar{y}$
2	$x\,\bar{y}$	$\bar{x} + y$
3	$x\,y$	$\bar{x} + \bar{y}$

Observations

- In the function tables:
  - Each minterm has one and only one 1 present in the 2^n terms (a minimum of 1s). All other entries are 0.
  - Each maxterm has one and only one 0 present in the 2^n terms All other entries are 1 (a maximum of 1s).
- We can implement any function by "ORing" the minterms corresponding to "1" entries in the function table. These are called the minterms of the function.
- We can implement any function by "ANDing" the maxterms corresponding to "0" entries in the function table. These are called the maxterms of the function.
- This gives us two canonical forms:
  - Sum of Minterms (SOM)
  - Product of Maxterms (POM)

  for stating any Boolean function.

Sum of Minterms (SOM)

- Example: Find $F_1 = m_1 + m_4 + m_7$
- $F1 = \overline{x}\ \overline{y}\ z + x\ \overline{y}\ \overline{z} + x\ y\ z$

x y z	index	m1	+	m4	+	m7	= F1
0 0 0	0	0	+	0	+	0	= 0
0 0 1	1	1	+	0	+	0	= 1
0 1 0	2	0	+	0	+	0	= 0
0 1 1	3	0	+	0	+	0	= 0
1 0 0	4	0	+	1	+	0	= 1
1 0 1	5	0	+	0	+	0	= 0
1 1 0	6	0	+	0	+	0	= 0
1 1 1	7	0	+	0	+	1	= 1

Product of Maxterms (POM)

- Example: Implement F1 in maxterms:

$F_1 = \quad M_0 \cdot M_2 \cdot M_3 \cdot M_5 \cdot M_6$

$$F_1 = (x + y + z) \cdot (x + \bar{y} + z) \cdot (x + \bar{y} + \bar{z}) \cdot (\bar{x} + y + \bar{z}) \cdot (\bar{x} + \bar{y} + z)$$

x y z	i	$M_0 \cdot M_2 \cdot M_3 \cdot M_5 \cdot M_6$ = F1
0 0 0	0	0 · 1 · 1 · 1 · 1 = 0
0 0 1	1	1 · 1 · 1 · 1 · 1 = 1
0 1 0	2	1 · 0 · 1 · 1 · 1 = 0
0 1 1	3	1 · 1 · 0 · 1 · 1 = 0
1 0 0	4	1 · 1 · 1 · 1 · 1 = 1
1 0 1	5	1 · 1 · 1 · 0 · 1 = 0
1 1 0	6	1 · 1 · 1 · 1 · 0 = 0
1 1 1	7	1 · 1 · 1 · 1 · 1 = 1

Canonical Sum of Minterms

- Any Boolean function can be expressed as a Sum of Minterms.
  - For the function table, the minterms used are the terms corresponding to the 1's
  - For expressions, expand all terms first to explicitly list all minterms. Do this by "ANDing" any term missing a variable v with a term ($v+\bar{v}$).
- Example: Implement $f = x + \bar{x}\ \bar{y}$ as a sum of minterms.

  First expand terms: $f = x(y+\bar{y}) + \bar{x}\ \bar{y}$

  Then distribute terms: $f = xy + x\bar{y} + \bar{x}\,\bar{y}$

  Express as sum of minterms: $f = m_3 + m_2 + m_0 = \Sigma_m(0, 2, 3)$

Canonical Product of Maxterms

- Any Boolean Function can be expressed as a Product of Maxterms (POM).
  - For the function table, the maxterms used are the terms corresponding to the 0's.
  - For an expression, expand all terms first to explicitly list all maxterms. Do this by first applying the second distributive law , "ORing" terms missing variable v with a term equal to $v \times \bar{v}$ and then applying the distributive law again.
- Example: Convert to product of maxterms:

$$f(x, y, z) = x + \bar{x}\,\bar{y}$$

Apply the distributive law:

$$x + \bar{x}\,\bar{y} = (x + \bar{x})(x + \bar{y}) = 1 \times (x + \bar{y}) = x + \bar{y}$$

Add missing variable z:

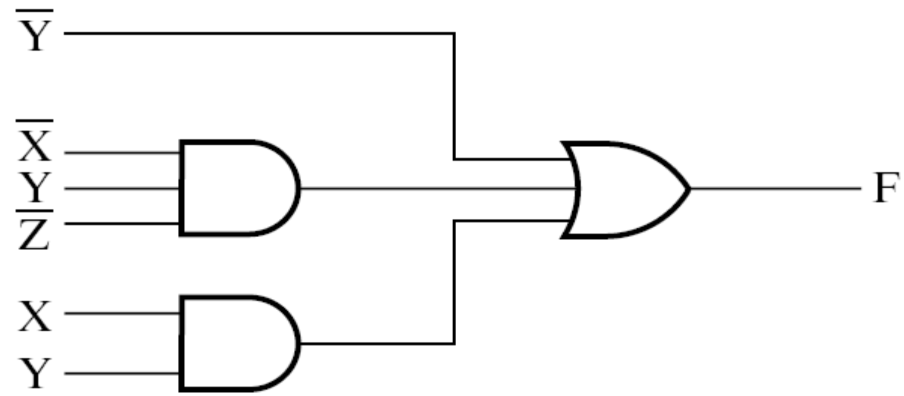
$$x + \bar{y} + z \times \bar{z} = (x + \bar{y} + z)\,(x + \bar{y} + \bar{z})$$

Express as POM: $f = M_2 \cdot M_3 = \Pi_M(2, 3)$

Sum of Products, Products of Sum

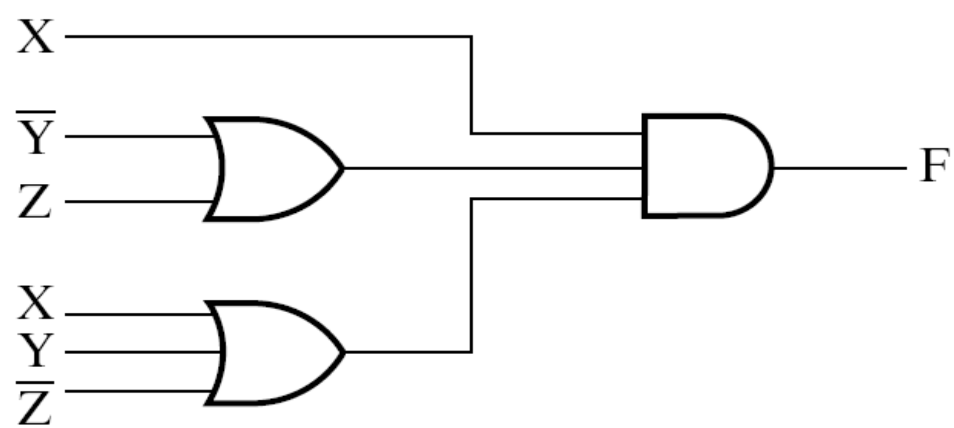
- Sum of products

$$F = \overline{Y} + \overline{X}Y\overline{Z} + XY$$



- Product of sums

$$F = X\left(\overline{Y} + Z\right)\left(X + Y + \overline{Z}\right)$$



Function Complements

- The complement of a function expressed as a sum of minterms is constructed by selecting the minterms missing in the sum-of-minterms canonical forms.
- Alternatively, the complement of a function expressed by a Sum of Minterms form is simply the Product of Maxterms with the same indices.
- Example: Given $F(x,y,z) = \Sigma_m(1,3,5,7)$

$$\overline{F}(x,y,z) = \Sigma_m(0,2,4,6)$$
$$\overline{F}(x,y,z) = \Pi_M(1,3,5,7)$$

Standard Forms

- Standard Sum-of-Products (SOP) form: equations are written as an OR of AND terms
- Standard Product-of-Sums (POS) form: equations are written as an AND of OR terms
- Examples:
  - SOP: $A\,B\,C + \overline{A}\,\overline{B}\,C + B$
  - POS: $(A+B)\cdot(A+\overline{B}+\overline{C})\cdot C$
- These "mixed" forms are neither SOP nor POS
  - $(A\,B + C)\,(A + C)$
  - $A\,B\,\overline{C} + A\,C\,(A + B)$

Standard Sum-of-Products (SOP)

- A sum of minterms form for *n* variables can be written down directly from a truth table.
  - Implementation of this form is a two-level network of gates such that:
  - The first level consists of *n*-input AND gates, and
  - The second level is a single OR gate (with fewer than 2^n inputs).
- This form often can be simplified so that the corresponding circuit is simpler.

Standard Sum-of-Products (SOP)

- A Simplification Example:

$$F(A,B,C) = \Sigma m(1,4,5,6,7)$$

- Writing the minterm expression:

$$F = \overline{A}\,\overline{B}\,C + A\,\overline{B}\,\overline{C} + A\,\overline{B}\,C + AB\overline{C} + ABC$$

- Simplifying:

$$\begin{aligned} F &= A'\,B'\,C + A\,(B'\,C' + B\,C' + B'\,C + B\,C) \\ &= A'\,B'\,C + A\,(B' + B)\,(C' + C) \\ &= A'\,B'\,C + A \cdot 1 \cdot 1 \\ &= A'\,B'\,C + A \\ &= B'C + A \end{aligned}$$

- Simplified F contains 3 literals compared with 15 in minterm F

SOP and POS Observations

- The previous examples show that:
  - Canonical Forms (Sum-of-minterms, Product-of-Maxterms), or other standard forms (SOP, POS) differ in complexity
  - Boolean algebra can be used to manipulate equations into simpler forms.
  - Simpler equations lead to simpler two-level implementations
- Questions:
  - How can we attain a “simplest” expression?
  - Is there only one minimum cost circuit?
  - The next part will deal with these issues.

Home Assignment

- 2-1a; 2-2a, c; 2-3a, c; 2-6b, d; 2-10a, c; 2-11a, c, d; 2-12b; 2-15; 2-17; 2-19a; 2-22a; 2-25b; 2-29; 2-30; 2-31;


Thank you!